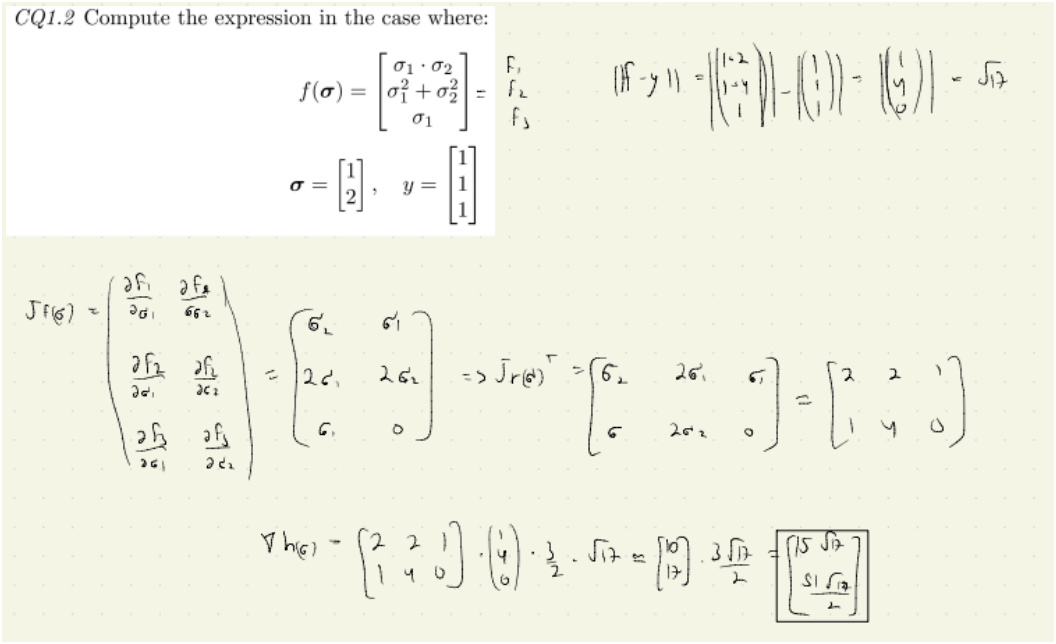
1 Part A: Calculus

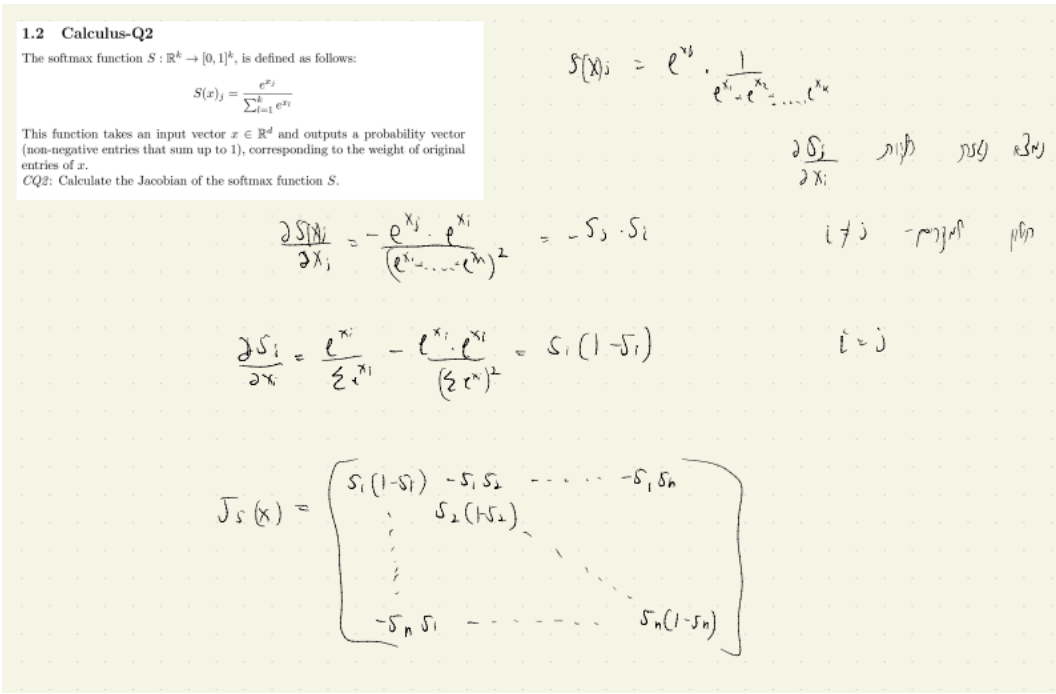
1.1 Calculus-Q1

CQ1.1 Use the chain rule to calculate the gradient of

$$h(\sigma) = \frac{1}{2}\|f(\sigma) - y\|^3$$

where $\sigma \in \mathbb{R}^m$ and f is some arbitrary function from $\mathbb{R}^m$ to $\mathbb{R}^n$.





Scenario 1 Results

Approximation Error (True Risk of best prophet): 0.2000
Average Error of selected prophets (over 100 experiments): 0.2600
Number of times the best prophet was chosen: 70 out of 100
Mean Estimation Error: 0.0600

This scenario had **two prophets** (True Risks: 0.20 and 0.40) competing on a training set of **one game**.

1. **Approximation Error (App Err$):**
    - **Calculation:** This is the minimum True Risk available: App Err = min(0.20, 0.40) = 0.20. This value is the best possible outcome for the system.
2. **Average Error (Avg Err):**
    - **Calculation:** This is the mean of the True Risks of the 100 prophets **selected** by the ERM process. Because m=1 caused high variance, the ERM often chose the 0.40 prophet by mistake.
    - **Result:** The Average Error (0.2600) is much higher than the best possible error (0.2000).
3. **Mean Estimation Error :**
    - **Calculation:** Est Err = Avg Err - App Err.
    - **Result:** The value 0.0600 shows the large penalty paid for relying on a single, unreliable game to make the model selection.

Scenario 2 Results:
Approximation Error: 0.2000
Average Error of selected prophets (N=10 experiments): 0.2280
Number of times the best prophet was chosen: 86 out of 100
Mean Estimation Error: 0.0280

This scenario had the **same two prophets as scenario 1** but increased the training set size to **ten games**.

- **Core Calculations are Identical:** The calculation methods for the Approximation Error, Average Error, and Estimation Error are **exactly the same**.
- **Key Difference:** Increasing the training size to m=10 significantly **reduced the Estimation Variance**. This allowed the ERM to calculate the training error more reliably.
- **Resulting Change:** The algorithm chose the best prophet (0.20 risk) much more frequently (86out of 100 times), causing the **Mean Estimation Error to drop sharply** from 0.0600 down to 0.0280. This demonstrates the immediate benefit of using more data to ensure a stable model selection.

The primary factor contributing to the observed change is the **decrease in the variance** of the Empirical Risk estimator, which is a direct consequence of increasing the sample size (N). This led to a more reliable **Empirical Risk Minimization** selection and a significant **reduction in the Mean Estimation Error**.
More times picking the best prophet.

Scenario 3 Results:
Approximation Error: 0.0064
Average Error of selected prophets: 0.0927
Mean Estimation Error: 0.0863
Number of times the BEST prophet was chosen: 1 out of 100
Number of times a prophet NOT 1% worse than the best was chosen: 5 out of 100

Across all scenarios, the three reported metrics are calculated by averaging results over 100 simulated trials, using the known **True Risk** of each prophet.

1. Approximation Error

This value defines the best-case performance for the current group of prophets (k).

- **How it was calculated:** The code finds the **minimum** value within the array of True Risks for the available prophet subset: App Err = min(Ltrue). This value is determined only by the hypothesis class and remains constant for all m values in a given row.

2. Average Error

This measures the expected error of the model actually chosen by the learning process.

- **How it was calculated:** In each of the 100 trials, the code retrieves the **True Risk** of the prophet selected by **Empirical Risk Minimization (ERM)**. These 100 recorded True Risks are then averaged (Avg Err = mean (Lf1, Lf2…).

3. Estimation Error

This measures the penalty incurred due to having a finite training set (m).

- **How it was calculated:** It is calculated as the simple difference: Eest = Avg Err - App Err. A high positive value indicates that the small training sample caused the ERM to frequently choose a suboptimal prophet.

These metrics measure the frequency of "success" in selecting a quality model, based on the **True Risk** of the chosen prophet in each of the 100 trials.

4. Times the Best Prophet Was Chosen

This metric checks how often the learning algorithm made the single most accurate selection possible.

- **Calculation:** In each of the 100 trials, the code checks if the **ID** of the prophet selected by **ERM** matches the **ID** of the prophet with the absolute lowest True Risk (calculated as np.argmin(Ltrue). A counter increments every time the IDs match.
- **Purpose:** Measures the algorithm's ability to perfectly distinguish the models despite sampling variability.

5. Times Not 1\% Worse Than the Best

This metric checks how often the algorithm made a selection that was "good enough" (within a tight tolerance of the optimum).

- Calculation: In each of the 100 trials, the code performs a check against the Approximation Error: Count if: Lselected <= App Err + 0.01 (Where Lselected is the True Risk of the chosen prophet).
- **Purpose:** Measures the practical success rate. A high count here shows that even if the algorithm didn't pick the *single best* prophet, it consistently chose one that was *practically optimal*.

1. Approximation Error
The **Approximation Error** is the best possible error rate available in the entire set of prophets.

- **Result: No Change.** The stays extremely low (near 0) because the best possible prophet in both ranges still has a True Risk near 0.
- **Easy Takeaway:** We haven't changed the quality of the absolute best model available; we've only removed all the really terrible models from the pool. Our models still have **low bias**.

2. Mean Estimation Error
The **Mean Estimation Error** is the penalty we pay because our small training set (N=10) causes us to frequently choose the wrong prophet (overfitting).

- **Result: Decreases.** This error drops significantly because the **worst possible mistake** we can make is now choosing a prophet with a True Risk of 0.5, not 1.0.
- Even when the system overfits (a high-variance mistake), the chosen model's True Risk is capped at 0.5. This **limits the magnitude of the resulting penalty** from the overfitting error, making the overall system more reliable.

In summary by improving the worst models in the pool, we reduce the cost of our algorithm's mistakes (the Estimation Error), even though the algorithm is just as likely to choose the wrong model (the complexity of the learning challenge remains the same).

<u>Scenario 4: Many Prophets, 1000 Games</u>
Approximation Error: 0.0064
Average Error of selected prophets: 0.0070
Mean Estimation Error: 0.0006
Number of times the BEST prophet was chosen: 56 out of 100
Times NOT 1% worse than the best: 100 out of 100

The calculation methods for the Approximation Error, Average Error, and Estimation Error, as well as the counting metrics (Times Best Chosen and Times Not 1% Worse), follow the **exact same logical process** described above for Scenario 3, using the relevant subset data and m games for each trial.

In Scenario 4 (N=1000), the generalization gap is **Near Zero**.

The gap closes because the large sample size eliminates **estimation variance**.

1. **Accurate Estimation:** N=1,000 is enough data for the **Training Error** to become an almost perfect score of the model's **True Error**.
2. **No Overfitting:** Since the training score is highly reliable, the learning algorithm correctly identifies models with genuinely low True Risk.
3. **Result:** The model performs just as well on the unseen test set as it did on the training set, meaning the difference between the two errors vanishes.

Scenario 5: School of Prophets

The calculation methods for the Approximation Error, Average Error, and Estimation Error in his Scenario follow the **exact same fundamental logic** as described above for Scenario 3.

Key for each cell: Average Error | Approximation Error | Estimation Error

k \ m	m=1 (N=1)	m=10 (N=10)	m=50 (N=50)	m=1000 (N=1000)
k=2	0.1032 0.0662 0.0370	0.0851 0.0662 0.0189	0.0728 0.0662 0.0066	0.0662 0.0662 0.0000
k=5	0.0937 0.0213 0.0723	0.0551 0.0213 0.0338	0.0328 0.0213 0.0115	0.0213 0.0213 0.0000
k=10	0.1157 0.0165 0.0992	0.0833 0.0165 0.0668	0.0260 0.0165 0.0096	0.0165 0.0165 0.0000
k=50	0.1033 0.0022 0.1012	0.0787 0.0022 0.0765	0.0186 0.0022 0.0164	0.0022 0.0022 0.0001

Explaining Observed Patterns in Scenario 5

The table illustrates the **Bias-Complexity Tradeoff** where the Model Pool (k) and Data Size (m) interact to determine performance.

Pattern 1: Moving Down the Rows (Increasing k)

- **Observation:** The **Approximation Error (App Err)** consistently drops (k=50 reaches aprrox 0.0022).
- **Link to Scenarios:** This mirrors the benefit of complexity seen in **Scenario 3/4** compared to **Scenario 1/2** (App Err fixed at 0.20). Increasing k means the class is less biased, guaranteeing access to a near-perfect model.

Pattern 2: Moving Across the Columns (Increasing m)

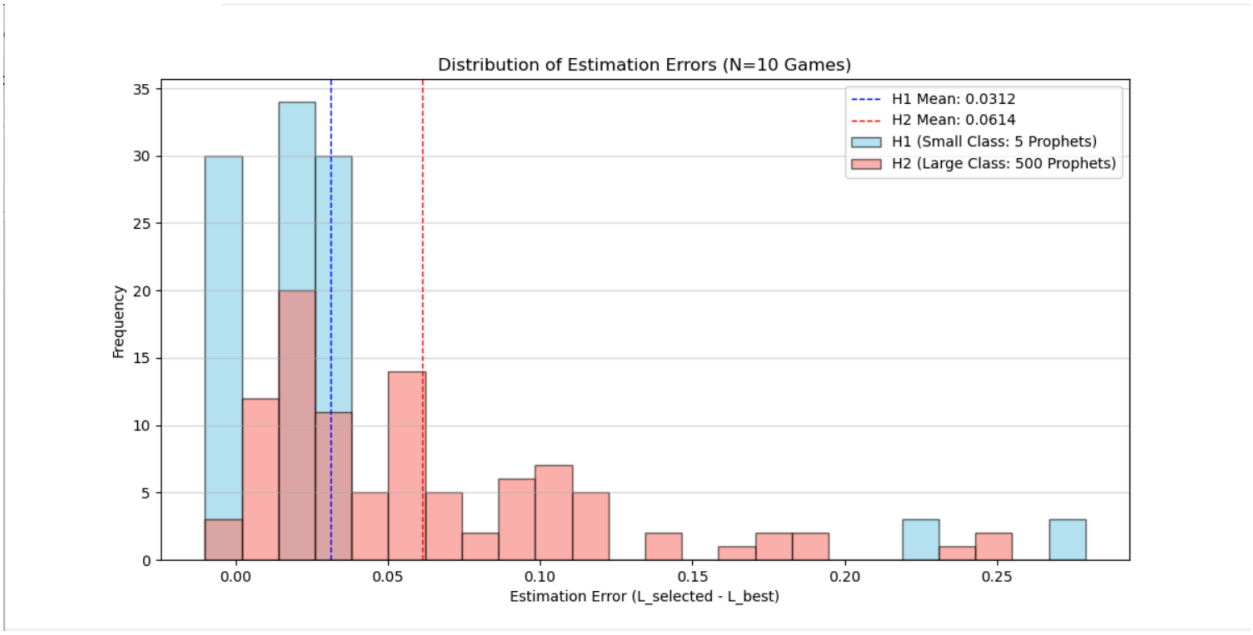
- **Observation:** The **Estimation Error (Est Err)** universally drops toward zero as m increases.
- **Link to Scenarios:**
    - **High Est Err (Low m):** The errors in the m=1 and m=10 columns reflect the **high variance** and **overfitting** problems observed in **Scenario 1** and **Scenario 3**.
    - **Zero Est Err(High m):** The error disappears entirely at m=1000 (like in **Scenario 4**), proving that sufficient data cures the sampling variance problem.

Conclusion on Performance

Optimal performance (Avg Err) is achieved in the **bottom-right corner** (k=50, m=1000), where the system benefits from the **low bias** of high complexity and the **low variance** of large data simultaneously.

Scenario 6:

Metric	Hypothesis 1 (H1)	Hypothesis 2 (H2)
Approx. Error (Bias)	0.3230	0.2500
Avg Test Error (Q1)	0.3542	0.3114
Avg Estimation Error (Q2)	0.0312	0.0614



Calculation Summary

All metrics are calculated by averaging the True Risk (Ltrue) results over 100 experiments:

- **Approximation Error (App Err):** Calculated as the **minimum** True Risk available in that specific hypothesis class (min(Ltrue)). (H1 approx 0.30, H2 approx 0.25).
- **Average Test Error:** Calculated as the **mean** of the True Risks of the 100 prophets selected by **ERM**.
- **Average Estimation Error (Est Err):** Calculated as the difference: Avg Test Err- App Err. This is the penalty paid for inaccurate selection.

Plotting and Tradeoff Interpretation

The histogram plots the distribution of the 100 **Individual Estimation Errors** for both classes using a common range and number of bins to ensure a direct comparison.

Class	Bias/Complexity	Observed Plot Pattern	Tradeoff Conclusion
Hypothesis 1 (H1)	Low Complexity / High Bias	**Tighter, lower distribution** (Mean approx 0.04).	**Stable:** Its small size makes the ERM decision reliable, resulting in a low **Estimation Error (low variance)**.
Hypothesis 2 (H2)	High Complexity / Low Bias	**Wider, higher distribution** (Mean approx 0.12).	**Unstable:** Its large size leads to extreme **overfitting** on N=10, causing large penalties (high variance) that override its theoretical low bias benefit.

The plot demonstrates that with small data, the high **variance** of the complex class (H2) makes it perform *worse* overall than the stable, simple class (H1), despite H2 having a better theoretical **bias** ceiling.

PAC Learning Analysis

In the **PAC (Probably Approximately Correct)** learning framework, a **hypothesis class** $\mathcal{H}$ denotes the set of all possible predictors (or "prophets") that the learning algorithm can choose from. Each hypothesis $h \in \mathcal{H}$ represents one possible rule or model mapping inputs to outputs.

Consider a hypothesis class with $|\mathcal{H}| = 100$, desired accuracy $\epsilon = 0.05$, and confidence level $\delta = 0.01$.

1. Compute the minimal number of samples m required for PAC learnability under these parameters.

According to the theorem

$$n \geq \frac{2\log(2n/\delta)}{\epsilon}$$

$$n \geq \frac{2\log(2\cdot 100/0.01)}{0.05} \Rightarrow n \geq 396$$

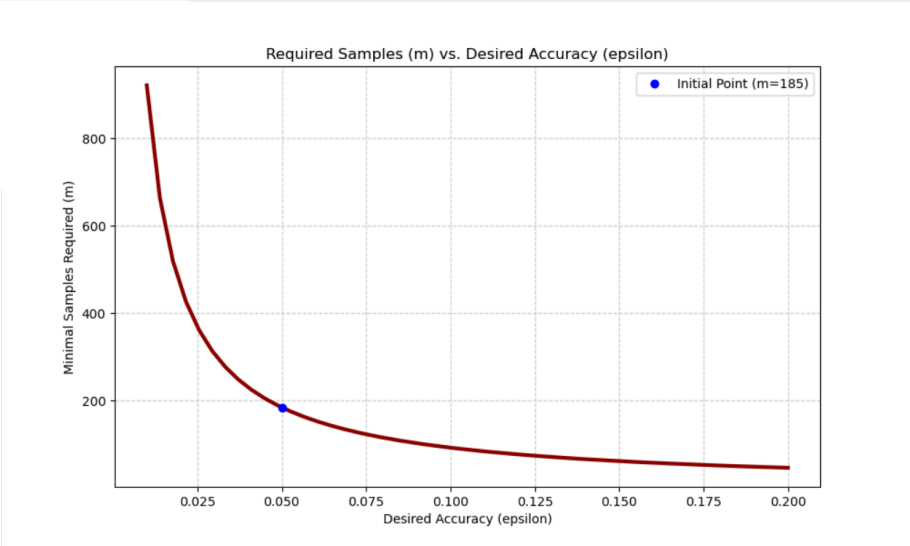
2. Analyze how m changes when the number of hypotheses (prophets) is doubled ($|\mathcal{H}| = 200$).

Just like the calculation before

$$n \geq \frac{2\log(2\cdot 200/0.01)}{0.05} \Rightarrow n \geq 423$$

even though the hypotheses doubled, the increase is relatively small and that is because its logarithmic

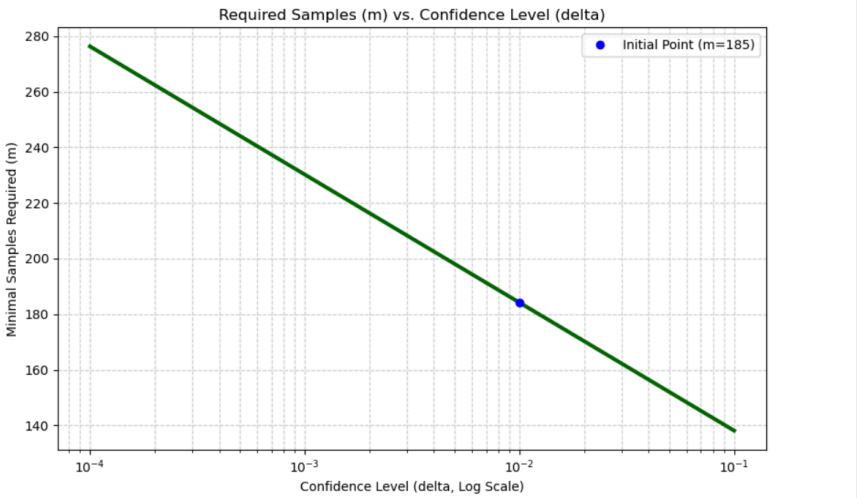
3)



The graph shows a sharp, **inverse relationship** between the desired accuracy (**ϵ**) and the minimum sample size (m).

- **Key Takeaway:** The required sample size m is proportional to 1/**ϵ**.
- **Analysis:** As you demand higher accuracy (ϵ approaches 0), the required number of samples grows extremely fast. For example, tightening the accuracy requirement from ϵ=0.1 to ϵ=0.05 (halving the error tolerance) causes the required training data size to approximately **double**. This shows the high computational cost of seeking precise accuracy in PAC learning.

4)



The graph shows a more gradual, **logarithmic relationship** between m and the required confidence (1/δ).

- **Key Takeaway:** The required sample size m is proportional to ln(1/δ).
- **Analysis:** As you demand higher confidence (δ gets smaller, moving left on the logarithmic axis), the sample size increases slowly. Increasing the confidence by a factor of 10 (δ=0.1 to δ=0.01) only requires a small, additive increase in m. This shows that the cost of achieving high confidence is much less demanding on data resources than the cost of achieving high accuracy (ϵ).

AI Tools

I used the AI tool in a few ways to help with this project. I used it to understand some of the harder questions in the assignment and give a description for my code.